

INTERNSHIP PROJECT REPORT

AI-Based Sentiment Analyzer Web Application

**Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science & Engineering
Batch: 2025-2029**

Submitted By:

Ashish Kumar Sinha (Roll No: 0519)
B.Tech CSE, Section 2CSE6

Host Institution / Repository Workspace:

GitHub Academic Workspace

1. Executive Summary

This internship report details the engineering, deployment, and hosting workflow of an **AI-Based Sentiment Analyzer** cloud application. Developed using Python, Flask, and Natural Language Processing libraries, the system parses complex textual feedback to classify human sentiment into quantitative polarities. The project successfully implements cloud architecture principles by bridging local developer silos to a highly available production environment on Render.com, integrated tightly with GitHub for version control and continuous deployment pipelines.

2. Introduction & Project Objectives

In modern data-driven architectures, unstructured text data is generated exponentially through product reviews, social channels, and client feedback. Manual classification is unscalable. The objectives of this industrial internship project include:

- **Automation:** Develop an automated lexical scanning service capable of judging text data streams in milliseconds.
- **Metric Isolation:** Deconstruct text segments into distinct numeric indicators representing emotional polarity and factual subjectivity.
- **High Availability Deployment:** Package project dependencies using web server gateways (Gunicorn) to ensure persistent uptime on the cloud.

3. System Technical Architecture

The platform is built using a decoupled modular architecture categorized across three specific core layers:

Layer Name	Tech Stack Utilized	Core Engineering Function
Frontend Interface	HTML5, CSS3, Bootstrap 5	Renders fluid, responsive dashboard UI accepting live user inputs.
Application Engine	Flask Framework, Python 3	Controls micro-routing, manages environment APIs, and calculates application parameters.
NLP Intelligence	TextBlob Engine, NLTK Corpora	Runs tokenization, applies lexical weights, and generates core metadata.
Cloud Server Layer	Render Linux Server, Gunicorn WSGI	Hosts application runtime, manages active ports, and processes web requests.

4. Algorithm & Computational Framework

The core NLP layer leverages **TextBlob** for semantic tracking, generating two distinct data structures:

- **Polarity Coefficient:** A continuous numeric float stretching between **-1.0 (Highly Negative)** to **+1.0 (Highly Positive)**. Text measuring exactly 0.00 signifies absolute neutrality.
- **Subjectivity Coefficient:** A scale bounding from **0.0 (Objective/Factual)** to **1.0 (Highly Subjective/Personal Opinion)**.

5. Cloud Infrastructure Setup & Packaging Issues Resolved

Migrating the system from an isolated local laptop runtime environment to an online scalable production server required engineering two mandatory configuration assets:

5.1 Web Gateway Integration (requirements.txt)

Standard internal Flask development servers are unstable for handling multi-tenant production traffic. The inclusion of the **Gunicorn** Web Server Gateway Interface (WSGI) was implemented to handle active production routing:

```
Flask==3.1.3
TextBlob==0.20.1
NLTK==3.10.3
gunicorn
```

5.2 Automation Shell Pipeline (build.sh)

Cloud runtime platforms compile binaries inside clean, isolated containers that initially lack necessary linguistic datasets. An automated build script was generated to download the foundational NLTK corpora during compile time:

```
#!/usr/bin/env bash
set -o errexit
pip install -r requirements.txt
python -m textblob.download_corpora
```

6. Deployment Footprint & Institutional References

The project has been compiled, pushed to configuration trees, and successfully deployed to the cloud infrastructure via isolated working directories.

Operational Artifacts:

- **Public Version Control Hub:**

https://github.com/ashish3445/2025-2029_AshishKumarSinha_0519_03sem_2CSE6

7. Conclusion

The practical execution of this assignment demonstrates the successful deployment of a machine learning workflow into a functional multi-user cloud context. By abstracting computational infrastructure, the platform establishes clear frameworks for scaling deep neural metrics globally.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.